```
public DateTime Date { get; set; }
public void OnGet(string cityName, int arrivalYear, int arrivalMonth, int
arrivalDay)
{
    Date = new DateTime(arrivalYear, arrivalMonth, arrivalDay);
}
```

A potential problem arises because there is no type of checking being performed against the incoming values. As far as routing is concerned, all URL data is a string type. Consequently, a request for /City/London/foo-bar-baz will match the route and result in an exception being raised when .NET attempts to construct a valid date in the year `foo`. Route constraints provide a solution to this problem. They enable you to specify the data type and range of acceptable values a route data item must comply with for it to be considered a match against the route template.

Route constraints are applied by separating them from the parameter name with a colon. The following example shows how to constrain each of the date parts to integer types:

```
"{cityName}/{arrivalYear:int}-{arrivalMonth:int}-{arrivalDay:int}"
```

The URL City/London/foo-bar-baz no longer matches this template, but /City/ London/2022-4-18 does. However, the user can still fiddle with the URL and provide values that result in an invalid date being created, such as /City/London/2022-4004-18. To protect against this, you could use the `range` constraint to constrain the range of valid values. The `range` constraint accepts minimum and maximum acceptable values and can be applied to the month parameter as follows:

```
"{cityName}/{arrivalYear:int}-{arrivalMonth:range(1-12)}-{arrivalDay:int}"
```